Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Отчёт по лабораторной работе № 2

по дисциплине «Операционные системы»

LNX-4 VTKM: вывод `struct task_struct` через системный вызов

(демонстрационное решение)

Выполнил:
студент группы P3331
Данилов Тимофей Николаевич

Преподаватель практики:
Максимов Андрей Николаевич

Санкт-Петербург, 2026

Содержание

1 Задание и вариант 2
2 Архитектура решения 2
3 Окружение 3
4 Почему понадобился патч ядра 3
5 Ход работы 4
6 Реализация 4
6.1 Патч ядра 4
6.2 Модуль 5
6.3 Программа пользователя 7
7 Сборка и запуск 7
8 Тестирование 7
8.1 Сверка с `/proc/1/status` 7
8.2 Процесс с `nice 5` 8
8.3 Поток ядра и ошибки 8
8.4 Автоматические проверки 9
9 Выводы 10
10 Приложение А. Листинги 11
10.1 `src/vtkm_info.h` 11
10.2 `src/vtkm.c` 11
10.3 `src/vtkm-cli.c` 13
10.4 Патч ядра 15
10.5 `Makefile` 17
10.6 `test.sh` 17
11 Приложение Б. Использование LLM 19

1 Задание и вариант

Разработать комплекс программ на уровне пользователя и на уровне ядра. Программа пользователя получает аргумент командной строки (не адрес), по которому ядро через свои таблицы находит целевую структуру, передаёт запрос в ядро и печатает ответ. Загружаемый модуль ядра принимает запрос через заданный интерфейс, находит структуру и возвращает данные пользователю.

Интерфейс	`syscall`: системный вызов
Целевая структура	задаётся именем из заголовков Linux; выбрана `struct task_struct` (`include/linux/sched.h`)
Аргумент программы	PID процесса или потока
Поиск в ядре	таблица PID: `find_get_pid` → `struct pid` → `get_pid_task` → `struct task_struct`

Это демонстрационное решение: оно сделано как пример для разбора порядка работы и не предназначено для сдачи (см. приложение Б). Код лежит в отдельном локальном репозитории `~/projects/vtkm-example` в виртуальной машине `vtkm`, порядок работы виден по истории коммитов.

2 Архитектура решения

```
./vtkm-cli 1234
   │  syscall(549, 1234, &info, sizeof info)
   ▼
ядро: SYSCALL_DEFINE3(vtkm, ...)        kernel/vtkm_syscall.c (патч ядра)
   │  под мьютексом вызывает обработчик, который зарегистрировал модуль;
   │  без модуля возвращает -ENOSYS
   ▼
модуль vtkm.ko: vtkm_handler(pid, buf, size)
   │  PID → struct pid → struct task_struct
   │  поля → struct vtkm_info → copy_to_user(buf, ...)
   ▼
vtkm-cli печатает поля
```

Листинг 1. Путь запроса от программы до модуля и обратно

Решение состоит из трёх частей:

- **патч ядра** — номер 549 в таблице системных вызовов и заглушка, которая передаёт вызов в модуль;
- **модуль `vtkm.ko`** — находит задачу по PID и копирует её поля пользователю;
- **программа `vtkm-cli`** — проверяет аргумент, делает системный вызов и печатает ответ.

Формат ответа `struct vtkm_info` описан в общем заголовке `src/vtkm_info.h`, который подключают и модуль, и программа. Поэтому раскладка полей у них одинаковая. Заголовок использует типы `__u64`, `__s32` из `<linux/types.h>`: в модуле

он раскрывается в заголовок ядра, в программе — в UAPI-заголовок `/usr/include/linux/types.h`, оба определяют эти типы одинаково.

3 Окружение

Хост	`home`: Ubuntu 24.04, Intel Core i5-11400F; на хосте включён Secure Boot, неподписанные модули не грузятся
Виртуальная машина	`vtkm` (libvirt/QEMU/KVM): Ubuntu 24.04, 8 vCPU, 8 ГБ, BIOS без Secure Boot
Ядро	Linux 6.18 из исходников (`~/linux-6.18`, ветка `vtkm-example`) с патчем из этой работы
Компилятор	GCC из Ubuntu 24.04, сборка модуля через kbuild

```
Linux vtkm 6.18.0+ #2 SMP PREEMPT_DYNAMIC Mon Nov  9 10:13:43 UTC 2026 x86_64 x86_64
x86_64 GNU/Linux
```

Модуль загружается только в виртуальной машине: ошибка в коде ядра роняет всю систему, а при сбое ВМ сообщения ядра остаются доступны через последовательную консоль (`console=ttyS0`, `virsh console vtkm`).

4 Почему понадобился патч ядра

Загружаемый модуль не может сам добавить системный вызов в современное ядро x86-64:

- `sys_call_table` не экспортируется для модулей;
- с версии 5.7 не экспортируется и `kallsyms_lookup_name`, через которую таблицу находили;
- с 5.3 на x86 запись в регистр CR0 закреплена (CR pinning), снять защиту страницы от записи нельзя;
- с 6.9 x86-64 вызывает обработчики через `switch` в `x64_sys_call`, а не через таблицу, поэтому подмена слота в `sys_call_table` ни на что не влияет.

Рассмотренные варианты:

Способ	**Плюсы**	**Минусы**
A. Заглушка в ядре + обработчик в модуле (**выбран**)	работает на любом ядре; модуль пересобирается и перезагружается без пересборки ядра	ядро нужно один раз пересобрать; нужна защита от гонки с выгрузкой модуля
B. Весь вызов в ядре	проще всего	нет загружаемого модуля, задание не выполнено
C. Подмена слота `sys_call_table` из модуля	без пересборки ядра	на ядрах ⩾ 6.9 не работает; приём руткитов
D. Перехват через kprobe/ftrace	без пересборки ядра	зависит от деталей реализации ядра, хак

5 Ход работы

Каждый шаг — отдельный коммит, сообщение коммита объясняет, что и зачем сделано.

```
b56ba7f Заготовка модуля из lab/vtkm
f4e804c Модуль: найти task_struct по PID и напечатать в dmesg
d6526a0 Патч ядра: системный вызов vtkm (номер 549)
bcc33af Модуль: обработчик системного вызова vtkm
b7b693a Программа vtkm-cli: вызвать vtkm и напечатать ответ
7fe08da Тесты: test.sh
f4cba2d README: схема, файлы, сборка и запуск
```

Коммит	Что сделано
`b56ba7f`	Заготовка `Makefile` и `src/vtkm.c` из курса без изменений. Проверка: `make`, `insmod`, `dmesg`, `rmmod`.
`f4e804c`	Поиск `task_struct` по PID, переданному параметром модуля (`insmod vtkm.ko pid=1`), вывод полей в `dmesg`. Так поиск проверен до появления системного вызова.
`d6526a0`	Патч ядра: системный вызов 549, `vtkm_register`/`vtkm_unregister`. Ядро пересобрано и установлено.
`bcc33af`	Модуль регистрирует обработчик, заполняет `struct vtkm_info`, отдаёт её через `copy_to_user`. Параметр `pid` убран.
`b7b693a`	Программа `vtkm-cli`; `make` собирает и модуль, и программу.
`7fe08da`	Скрипт проверок `test.sh`.
`f4cba2d`	README: схема, файлы, сборка и запуск.

6 Реализация

6.1 Патч ядра

Патч (`kernel/0001-vtkm-syscall-549-handler-is-registered-by-a-module.patch`) накладывается на Linux 6.18 поверх учебного hello-вызова с номером 548 и добавляет:

- строку `549 common vtkm sys_vtkm` в `arch/x86/entry/syscalls/syscall_64.tbl`;
- `include/linux/vtkm.h` с типом обработчика и функциями регистрации;
- `kernel/vtkm_syscall.c` и строку `obj-y += vtkm_syscall.o` в `kernel/Makefile`: заглушка вшита в ядро.

```c
static DEFINE_MUTEX(vtkm_lock);
static vtkm_handler_t vtkm_handler;

SYSCALL_DEFINE3(vtkm, int, pid, void __user *, buf, size_t, size)
{
  long ret = -ENOSYS;

  mutex_lock(&vtkm_lock);
  if (vtkm_handler)
```

```
    ret = vtkm_handler(pid, buf, size);
  mutex_unlock(&vtkm_lock);
  return ret;
}
```

Защита от гонки с выгрузкой. Если бы заглушка просто читала указатель на обработчик, модуль можно было бы выгрузить, пока другой процесс ещё выполняет его код, и ядро обратилось бы к освобождённой памяти. Здесь мьютекс держится и во время вызова обработчика, а `vtkm_unregister` берёт тот же мьютекс. Поэтому `rmmod` ждёт, пока текущие вызовы выйдут из модуля, и только потом модуль выгружается. Цена — вызовы выполняются по одному, для этой задачи это не важно. Повторная регистрация возвращает `-EBUSY`. Функции регистрации экспортированы через `EXPORT_SYMBOL_GPL`.

После сборки и перезагрузки символы заглушки видны в ядре:

```
0000000000000000 T __pfx_vtkm_register
0000000000000000 T vtkm_register
0000000000000000 T __pfx_vtkm_unregister
0000000000000000 T vtkm_unregister
0000000000000000 T __pfx___ia32_sys_vtkm
0000000000000000 T __ia32_sys_vtkm
0000000000000000 T __pfx___x64_sys_vtkm
0000000000000000 T __x64_sys_vtkm
```

6.2 Модуль

Обработчик системного вызова:

```
static long vtkm_handler(int nr, void __user *buf, size_t size) {
  ...
  if (size != sizeof(info))
    return -EINVAL;

  p = find_get_pid(nr);               /* таблица PID, ссылка на struct pid +1 */
  if (p == NULL)
    return -ESRCH;
  task = get_pid_task(p, PIDTYPE_PID); /* ссылка на task_struct +1 */
  put_pid(p);
  if (task == NULL)
    return -ESRCH;

  memset(&info, 0, sizeof(info));
  fill_info(task, &info);
  put_task_struct(task);

  if (copy_to_user(buf, &info, sizeof(info)) != 0)
    return -EFAULT;
  return 0;
}
```

Важные места:

- **Счётчики ссылок.** `find_get_pid` и `get_pid_task` увеличивают счётчики ссылок, поэтому задача не может быть освобождена, пока модуль читает её поля. После использования ссылки отпускаются (`put_pid`, `put_task_struct`).
- **Память процесса.** `total_vm` берётся из `mm_struct` через `get_task_mm`/`mmput`. У потоков ядра `mm` нет, тогда поле остаётся 0.
- **Только значения.** Наружу копируется своя структура `vtkm_info`, а не `task_struct`: указатели ядра пользователю не отдаются, раскладка не зависит от версии ядра.
- **Обнуление.** `memset` перед заполнением не даёт утечь содержимому стека ядра через байты выравнивания.
- **`copy_to_user`.** Указатель пользователя не разыменовывается напрямую: функция проверяет адрес и обрабатывает отсутствующую страницу, возвращая ошибку вместо oops.

Поле `vtkm_info`	Источник	Смысл
`comm`	`get_task_comm`	имя программы
`pid`, `tgid`	`task->pid`, `task->tgid`	id потока и id процесса
`ppid`	`task_ppid_nr`	id родителя
`state`	`task_state_to_char`	символ состояния, как в `/proc/<pid>/status`
`prio`, `static_prio`	`task->prio`, `task->static_prio`	приоритет; `static_prio = 120 + nice`
`policy`	`task->policy`	политика планирования
`nr_threads`	`get_nr_threads`	число потоков процесса
`utime_ns`, `stime_ns`	`task->utime`, `task->stime`	время в user и kernel mode, нс
`nvcsw`, `nivcsw`	`task->nvcsw`, `task->nivcsw`	добровольные и принудительные переключения
`total_vm`	`mm->total_vm`	размер виртуальной памяти в страницах

Коды ошибок:

Код	Когда
`-ENOSYS`	модуль не загружен (возвращает заглушка ядра)
`-EINVAL`	размер буфера не равен `sizeof(struct vtkm_info)`
`-ESRCH`	процесса или потока с таким PID нет
`-EFAULT`	неверный адрес буфера

Данные модуля:

```
filename:       /home/volin/projects/vtkm-example/src/vtkm.ko
license:        GPL
depends:
name:           vtkm
vermagic:       6.18.0+ SMP preempt mod_unload modversions
```

`depends` пуст: `vtkm_register` находится во вшитой части ядра, а не в другом модуле.

6.3 Программа пользователя

`vtkm-cli PID` проверяет, что аргумент — одно число от 1 до 4194304 (`pid_max`), вызывает `syscall(549, pid, &info, sizeof info)` и печатает поля по одному в строке. На `ENOSYS` подсказывает загрузить модуль, на `ESRCH` сообщает, что процесса нет. Код выхода: 0 — успех, 1 — ошибка ядра, 2 — неверный аргумент.

7 Сборка и запуск

Ядро (один раз):

```
cd ~/linux-6.18 && git am ~/projects/vtkm-example/kernel/0001-*.patch
make -j8 && sudo make modules_install install && sudo reboot
```

Модуль и программа:

```
cd ~/projects/vtkm-example
make                        # vtkm-cli и src/vtkm.ko
sudo insmod src/vtkm.ko
./vtkm-cli 1
sudo rmmod vtkm
./test.sh
```

8 Тестирование

8.1 Сверка с `/proc/1/status`

`./vtkm-cli 1`

```
struct task_struct of PID 1
  comm         systemd
  pid          1
  tgid         1
  ppid         0
  state        S
  prio         120
  static_prio  120 (nice 0)
  policy       0 (SCHED_OTHER)
  threads      1
  utime        693000000 ns
  stime        1130601219 ns
  nvcsw        4524
  nivcsw       910
  total_vm     5503 pages (22012 kB)
```

`/proc/1/status`

```
Name: systemd
State:  S (sleeping)
Tgid: 1
Pid:  1
PPid: 0
VmSize:    22012 kB
Threads:  1
voluntary_ctxt_switches:  4524
nonvoluntary_ctxt_switches: 910
```

Совпадают имя, состояние, PID, родитель, число потоков, переключения контекста (4524 и 910), размер виртуальной памяти: 5503 страницы × 4 КиБ = 22012 kB = `VmSize`. Функция ядра, которая формирует `/proc/<pid>/status` (`proc_pid_status` в `fs/proc/array.c`), читает те же поля `task_struct` и так же берёт `mm` через `get_task_mm`/`mmput`.

8.2 Процесс с `nice 5`

`nice -n 5 sleep 60 &`, затем `./vtkm-cli $!`:

```
struct task_struct of PID 7779
  comm         sleep
  pid          7779
  tgid         7779
  ppid         7561
  state        S
  prio         125
  static_prio  125 (nice 5)
  policy       0 (SCHED_OTHER)
  threads      1
  utime        0 ns
  stime        1000000 ns
  nvcsw        1
  nivcsw       0
  total_vm     1531 pages (6124 kB)
```

`static_prio = 120 + 5 = 125`, процесс спит (`S`), один поток.

8.3 Поток ядра и ошибки

`./vtkm-cli 2` (kthreadd, поток ядра, `mm` нет):

```
struct task_struct of PID 2
  comm         kthreadd
  pid          2
  tgid         2
  ppid         0
  state        S
  prio         120
  static_prio  120 (nice 0)
  policy       0 (SCHED_OTHER)
  threads      1
  utime        0 ns
  stime        2000000 ns
  nvcsw        274
  nivcsw       1
  total_vm     0 pages (0 kB)
```

Запуск	Вывод
модуль не загружен, `./vtkm-cli 1`	`vtkm: no handler, is module vtkm.ko loaded?`
`./vtkm-cli 4000000`	`vtkm: no process with PID 4000000`

`./vtkm-cli abc`	`bad PID: abc`

Журнал модуля за прогон:

```
[12745.114180] [vtkm]: VTKM joined the kernel
[12745.115342] [vtkm]: answered for pid 1
[12745.420114] [vtkm]: answered for pid 7779
[12745.422748] [vtkm]: answered for pid 2
[12745.437097] [vtkm]: VTKM left the kernel
```

8.4 Автоматические проверки

`test.sh` сам загружает и выгружает модуль и проверяет: ответ без модуля; совпадение `comm`, `ppid`, `state` для PID 1 с `/proc/1/status`; свой процесс `sleep` (имя, родитель — сам скрипт, состояние); `nice 10` → `static_prio 130`; поток ядра `kthreadd` с `total_vm = 0`; PID потока (`pid` — id потока, `tgid` — id процесса); неверные аргументы; ответ после `rmmod`.

```
OK   без модуля: ENOSYS и подсказка
OK   PID 1: comm
OK   PID 1: ppid
OK   PID 1: state
OK   sleep: comm sleep
OK   sleep: ppid = этот скрипт
OK   sleep: state S
OK   nice 10: static_prio 130
OK   PID 2: kthreadd, total_vm 0
OK   поток 2836: pid 2836, tgid 2835
OK   нет такого PID: код 1
OK   буквы вместо PID: код 2
OK   без аргумента: код 2
OK   PID 0: код 2
OK   после rmmod: код 1
passed 15, failed 0
```

Не проверены `-EINVAL` и `-EFAULT`: программа всегда передаёт правильный буфер, для проверки нужна отдельная тестовая программа с неверным размером и адресом.

После загрузки внешнего модуля ядро помечено флагами taint:

```
12288
```

12288 = 4096 + 8192: `O` (модуль собран вне дерева ядра) и `E` (модуль не подписан).

9 Выводы

- Модуль не может добавить системный вызов в ядро 6.18 на x86-64, поэтому решение разделено: номер вызова и тонкая заглушка вшиты в ядро, а логика находится в модуле и регистрируется при загрузке. Модуль можно пересобирать без пересборки ядра.
- Безопасная выгрузка модуля обеспечена мьютексом, который держится во время вызова обработчика: `rmmod` дожидается окончания текущих вызовов.
- Поиск задачи идёт через таблицу PID ядра с подсчётом ссылок, наружу копируется собственная структура только со значениями полей через `copy_to_user`.
- Вывод программы совпадает с `/proc/<pid>/status`; все 15 автоматических проверок проходят.
- В реальной практике эти данные берут из `/proc/<pid>/status` или через eBPF: новый системный вызов — постоянный ABI ядра, его добавляют только для новых фундаментальных возможностей.

10 Приложение А. Листинги

10.1 src/vtkm_info.h

```
/*
 * Формат ответа системного вызова vtkm.
 * Этот файл подключают и модуль ядра, и программа vtkm-cli,
 * поэтому раскладка полей у них одинаковая.
 */
#ifndef VTKM_INFO_H
#define VTKM_INFO_H

#include <linux/types.h>

#define VTKM_SYSCALL_NR 549
#define VTKM_COMM_LEN 16

/* Значения полей struct task_struct (указатели не отдаём) */
struct vtkm_info {
  char comm[VTKM_COMM_LEN]; /* имя программы */
  __s32 pid;                /* id потока */
  __s32 tgid;               /* id процесса (группы потоков) */
  __s32 ppid;               /* id родителя */
  __s32 state;              /* символ состояния: R, S, D, T, Z ... */
  __s32 prio;               /* текущий приоритет планировщика */
  __s32 static_prio;        /* приоритет от nice: 120 + nice */
  __u32 policy;             /* политика: 0 SCHED_OTHER, 1 FIFO, 2 RR ... */
  __s32 nr_threads;         /* число потоков в процессе */
  __u64 utime_ns;           /* время в user mode, нс */
  __u64 stime_ns;           /* время в kernel mode, нс */
  __u64 nvcsw;              /* добровольные переключения контекста */
  __u64 nivcsw;             /* принудительные переключения контекста */
  __u64 total_vm;           /* размер виртуальной памяти, страниц (0 у потоков ядра) */
};

#endif
```

10.2 src/vtkm.c

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/printk.h>
#include <linux/pid.h>
#include <linux/sched.h>
#include <linux/sched/mm.h>
#include <linux/sched/signal.h>
#include <linux/sched/task.h>
#include <linux/string.h>
#include <linux/uaccess.h>
#include <linux/vtkm.h>
```

```c
#include "vtkm_info.h"

#define MODULE_NAME "vtkm"

MODULE_LICENSE("GPL");
MODULE_AUTHOR("secs-dev");
MODULE_DESCRIPTION("A simple kernel module");

#define LOG(fmt, ...) pr_info("[" MODULE_NAME "]: " fmt, ##__VA_ARGS__)

/* Скопировать нужные поля task_struct в ответ */
static void fill_info(struct task_struct *task, struct vtkm_info *info) {
  struct mm_struct *mm;

  get_task_comm(info->comm, task);
  info->pid = task->pid;
  info->tgid = task->tgid;
  info->ppid = task_ppid_nr(task);
  info->state = task_state_to_char(task);
  info->prio = task->prio;
  info->static_prio = task->static_prio;
  info->policy = task->policy;
  info->nr_threads = get_nr_threads(task);
  info->utime_ns = task->utime;
  info->stime_ns = task->stime;
  info->nvcsw = task->nvcsw;
  info->nivcsw = task->nivcsw;

  /* У потоков ядра нет mm, тогда get_task_mm вернёт NULL */
  mm = get_task_mm(task);
  if (mm != NULL) {
    info->total_vm = mm->total_vm;
    mmput(mm);
  }
}

/* Обработчик системного вызова vtkm(pid, buf, size) */
static long vtkm_handler(int nr, void __user *buf, size_t size) {
  struct pid *p;
  struct task_struct *task;
  struct vtkm_info info;

  if (size != sizeof(info))
    return -EINVAL;

  /* PID -> struct pid (таблица PID ядра), счётчик ссылок +1 */
  p = find_get_pid(nr);
  if (p == NULL)
    return -ESRCH;

  /* struct pid -> struct task_struct, счётчик ссылок задачи +1 */
  task = get_pid_task(p, PIDTYPE_PID);
  put_pid(p);
  if (task == NULL)
    return -ESRCH;
```

```
  memset(&info, 0, sizeof(info));
  fill_info(task, &info);
  put_task_struct(task);

  if (copy_to_user(buf, &info, sizeof(info)) != 0)
    return -EFAULT;

  LOG("answered for pid %d\n", nr);
  return 0;
}

static int __init vtkm_init(void) {
  int ret;

  ret = vtkm_register(vtkm_handler);
  if (ret != 0) {
    LOG("can't register handler: %d\n", ret);
    return ret;
  }
  LOG("VTKM joined the kernel\n");
  return 0;
}

static void __exit vtkm_exit(void) {
  vtkm_unregister();
  LOG("VTKM left the kernel\n");
}

module_init(vtkm_init);
module_exit(vtkm_exit);
```

10.3 `src/vtkm-cli.c`

```
/*
 * vtkm-cli PID
 * Спрашивает у ядра через системный вызов vtkm поля struct task_struct
 * процесса PID и печатает их.
 */
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/syscall.h>
#include <unistd.h>

#include "vtkm_info.h"

static const char *policy_name(unsigned int policy) {
  switch (policy) {
  case 0: return "SCHED_OTHER";
  case 1: return "SCHED_FIFO";
  case 2: return "SCHED_RR";
```

```
  case 3: return "SCHED_BATCH";
  case 5: return "SCHED_IDLE";
  case 6: return "SCHED_DEADLINE";
  case 7: return "SCHED_EXT";
  default: return "?";
  }
}

int main(int argc, char *argv[]) {
  struct vtkm_info info;
  char *end;
  long pid;
  long ret;

  if (argc != 2) {
    fprintf(stderr, "usage: %s PID\n", argv[0]);
    return 2;
  }

  pid = strtol(argv[1], &end, 10);
  if (*argv[1] == '\0' || *end != '\0' || pid <= 0 || pid > 4194304) {
    fprintf(stderr, "bad PID: %s\n", argv[1]);
    return 2;
  }

  ret = syscall(VTKM_SYSCALL_NR, (int)pid, &info, sizeof(info));
  if (ret != 0) {
    if (errno == ENOSYS)
      fprintf(stderr, "vtkm: no handler, is module vtkm.ko loaded?\n");
    else if (errno == ESRCH)
      fprintf(stderr, "vtkm: no process with PID %ld\n", pid);
    else
      fprintf(stderr, "vtkm: %s\n", strerror(errno));
    return 1;
  }

  printf("struct task_struct of PID %ld\n", pid);
  printf("  comm         %s\n", info.comm);
  printf("  pid          %d\n", info.pid);
  printf("  tgid         %d\n", info.tgid);
  printf("  ppid         %d\n", info.ppid);
  printf("  state        %c\n", info.state);
  printf("  prio         %d\n", info.prio);
  printf("  static_prio  %d (nice %d)\n", info.static_prio, info.static_prio - 120);
  printf("  policy       %u (%s)\n", info.policy, policy_name(info.policy));
  printf("  threads      %d\n", info.nr_threads);
  printf("  utime        %llu ns\n", (unsigned long long)info.utime_ns);
  printf("  stime        %llu ns\n", (unsigned long long)info.stime_ns);
  printf("  nvcsw        %llu\n", (unsigned long long)info.nvcsw);
  printf("  nivcsw       %llu\n", (unsigned long long)info.nivcsw);
  printf("  total_vm     %llu pages (%llu kB)\n",
         (unsigned long long)info.total_vm,
         (unsigned long long)info.total_vm * (unsigned long long)getpagesize() / 1024);
  return 0;
}
```

10.4 Патч ядра

```
From cac2c45d1ecb5889b66052364305ecc48a96705c Mon Sep 17 00:00:00 2001
From: Timofey Danilov <volinadtim@gmail.com>
Date: Mon, 9 Nov 2026 10:07:31 +0000
Subject: [PATCH] vtkm: syscall 549, handler is registered by a module

---
 arch/x86/entry/syscalls/syscall_64.tbl |  1 +
 include/linux/vtkm.h                   | 13 +++++++
 kernel/Makefile                        |  1 +
 kernel/vtkm_syscall.c                  | 51 ++++++++++++++++++++++++++++
 4 files changed, 66 insertions(+)
 create mode 100644 include/linux/vtkm.h
 create mode 100644 kernel/vtkm_syscall.c

diff --git a/arch/x86/entry/syscalls/syscall_64.tbl b/arch/x86/entry/syscalls/
syscall_64.tbl
index 5952c43c9..b74985e72 100644
--- a/arch/x86/entry/syscalls/syscall_64.tbl
+++ b/arch/x86/entry/syscalls/syscall_64.tbl
@@ -440,4 +440,5 @@
 # This is the end of the legacy x32 range.  Numbers 548 and above are
 # not special and are not to be used for x32-specific syscalls.
 548     common  hello                   sys_hello
+549  common  vtkm      sys_vtkm

diff --git a/include/linux/vtkm.h b/include/linux/vtkm.h
new file mode 100644
index 000000000..f5997d3d2
--- /dev/null
+++ b/include/linux/vtkm.h
@@ -0,0 +1,13 @@
+/* SPDX-License-Identifier: GPL-2.0 */
+#ifndef _LINUX_VTKM_H
+#define _LINUX_VTKM_H
+
+#include <linux/types.h>
+
+/* Обработчик системного вызова vtkm, его даёт загружаемый модуль */
+typedef long (*vtkm_handler_t)(int pid, void __user *buf, size_t size);
+
+int vtkm_register(vtkm_handler_t handler);
+void vtkm_unregister(void);
+
+#endif
diff --git a/kernel/Makefile b/kernel/Makefile
index b492614e3..ff9c0d71a 100644
--- a/kernel/Makefile
+++ b/kernel/Makefile
@@ -208,3 +208,4 @@ $(obj)/kheaders.md5: $(obj)/kheaders-srclist FORCE
 clean-files := kheaders.md5 kheaders-srclist kheaders-objlist

 obj-y += hello_syscall.o
+obj-y += vtkm_syscall.o
```

```
diff --git a/kernel/vtkm_syscall.c b/kernel/vtkm_syscall.c
new file mode 100644
index 000000000..ce5c8d2d1
--- /dev/null
+++ b/kernel/vtkm_syscall.c
@@ -0,0 +1,51 @@
+// SPDX-License-Identifier: GPL-2.0
+/*
+ * Заглушка системного вызова vtkm. Сама ничего не делает: вызывает
+ * обработчик, который зарегистрировал модуль vtkm.ko. Без модуля -ENOSYS.
+ */
+#include <linux/errno.h>
+#include <linux/export.h>
+#include <linux/mutex.h>
+#include <linux/syscalls.h>
+#include <linux/vtkm.h>
+
+static DEFINE_MUTEX(vtkm_lock);
+static vtkm_handler_t vtkm_handler;
+
+int vtkm_register(vtkm_handler_t handler)
+{
+ int ret = 0;
+
+ mutex_lock(&vtkm_lock);
+ if (vtkm_handler)
+   ret = -EBUSY;
+ else
+   vtkm_handler = handler;
+ mutex_unlock(&vtkm_lock);
+ return ret;
+}
+EXPORT_SYMBOL_GPL(vtkm_register);
+
+/*
+ * Мьютекс держится и во время вызова обработчика, поэтому unregister
+ * ждёт, пока закончатся вызовы, которые уже внутри модуля. После этого
+ * модуль можно выгружать.
+ */
+void vtkm_unregister(void)
+{
+ mutex_lock(&vtkm_lock);
+ vtkm_handler = NULL;
+ mutex_unlock(&vtkm_lock);
+}
+EXPORT_SYMBOL_GPL(vtkm_unregister);
+
+SYSCALL_DEFINE3(vtkm, int, pid, void __user *, buf, size_t, size)
+{
+ long ret = -ENOSYS;
+
+ mutex_lock(&vtkm_lock);
+ if (vtkm_handler)
+   ret = vtkm_handler(pid, buf, size);
+ mutex_unlock(&vtkm_lock);
```

```
+ return ret;
+}
--
2.43.0
```

10.5 `Makefile`

```
obj-m += src/vtkm.o

PWD := $(CURDIR)
KDIR = /lib/modules/`uname -r`/build
EXTRA_CFLAGS = -Wall -g

all: vtkm-cli
  make -C $(KDIR) M=$(PWD) modules

vtkm-cli: src/vtkm-cli.c src/vtkm_info.h
  gcc -Wall -Wextra -o vtkm-cli src/vtkm-cli.c

clean:
  make -C $(KDIR) M=$(PWD) clean
  rm -rf .cache vtkm-cli
```

10.6 `test.sh`

```
#!/bin/bash
# Проверки vtkm. Запуск: make && ./test.sh
# Модуль скрипт загружает и выгружает сам (нужен sudo).

ok=0
fail=0

check() {
  # check "название" команда-проверка
  if eval "$2"; then
    echo "OK   $1"
    ok=$((ok + 1))
  else
    echo "FAIL $1"
    fail=$((fail + 1))
  fi
}

# Значение поля из вывода vtkm-cli: field PID имя
field() {
  ./vtkm-cli "$1" | awk -v f="$2" '$1 == f { print $2 }'
}

# Значение строки из /proc/PID/status: status PID Имя
status() {
  awk -v f="$2:" '$1 == f { print $2 }' "/proc/$1/status"
```

```
}

sudo rmmod vtkm 2>/dev/null

# 1. Без модуля заглушка ядра отвечает ENOSYS
check "без модуля: ENOSYS и подсказка" \
  '[ "$(./vtkm-cli 1 2>&1)" = "vtkm: no handler, is module vtkm.ko loaded?" ]'

sudo insmod src/vtkm.ko

# 2. PID 1 сверяем с /proc/1/status
check "PID 1: comm"  '[ "$(field 1 comm)" = "$(status 1 Name)" ]'
check "PID 1: ppid"  '[ "$(field 1 ppid)" = "$(status 1 PPid)" ]'
check "PID 1: state" '[ "$(field 1 state)" = "$(status 1 State)" ]'

# 3. Свой процесс: sleep в фоне
sleep 30 &
p=$!
sleep 0.2
check "sleep: comm sleep"     '[ "$(field $p comm)" = "sleep" ]'
check "sleep: ppid = этот скрипт" '[ "$(field $p ppid)" = "$$" ]'
check "sleep: state S"        '[ "$(field $p state)" = "S" ]'
kill $p

# 4. nice 10 даёт static_prio 130
nice -n 10 sleep 30 &
p=$!
sleep 0.2
check "nice 10: static_prio 130" '[ "$(field $p static_prio)" = "130" ]'
kill $p

# 5. Поток ядра kthreadd (PID 2): mm нет, total_vm 0
check "PID 2: kthreadd, total_vm 0" \
  '[ "$(field 2 comm)" = "kthreadd" ] && [ "$(field 2 total_vm)" = "0" ]'

# 6. PID потока: pid = id потока, tgid = id процесса
for d in /proc/[0-9]*/task/*; do
  t=${d##*/}
  pr=$(echo "$d" | cut -d/ -f3)
  if [ "$t" != "$pr" ]; then tid=$t; break; fi
done
check "поток $tid: pid $tid, tgid $pr" \
  '[ "$(field $tid pid)" = "$tid" ] && [ "$(field $tid tgid)" = "$pr" ]'

# 7. Ошибки
check "нет такого PID: код 1" './vtkm-cli 4000000 >/dev/null 2>&1; [ $? -eq 1 ]'
check "буквы вместо PID: код 2" './vtkm-cli abc >/dev/null 2>&1; [ $? -eq 2 ]'
check "без аргумента: код 2"   './vtkm-cli >/dev/null 2>&1; [ $? -eq 2 ]'
check "PID 0: код 2"           './vtkm-cli 0 >/dev/null 2>&1; [ $? -eq 2 ]'

# 8. После выгрузки снова ENOSYS
sudo rmmod vtkm
check "после rmmod: код 1" './vtkm-cli 1 >/dev/null 2>&1; [ $? -eq 1 ]'
```

```
echo "passed $ok, failed $fail"
[ $fail -eq 0 ]
```

11 Приложение Б. Использование LLM

Весь код этого демонстрационного решения (модуль, программа, патч ядра, тесты) и этот отчёт сгенерированы Claude Opus 5.5 (Claude Code) по просьбе студента как пример выполнения работы. Запрос: «сделай сейчас в качестве примера всю лабу сам в ВМ в отдельной папке в отдельном гит, веди корректно комментарии, чтобы я мог по их истории изменений понять порядок; решение делай максимально примитивным, минимально правильное и понятное».

По правилам курса ключевую логику лабораторной работы нельзя писать через LLM, поэтому это решение используется только для разбора и подготовки к защите и не сдаётся как собственная работа.